



Syntax Analysis (Part 2)



Top-Down Parsing

Top-down parsing is a strategy where the parser attempts to derive the input string from the Start Symbol of the grammar.

- **Direction of Derivation:** It finds the **Leftmost Derivation** of the input string.
- **Parse Tree Construction:** The parse tree is constructed from the Root (Start Symbol) down toward the Leaves (Input Terminals).
- **Core Idea:** The parser tries to predict or match the non-terminal symbols on the stack to the terminals in the input stream. It always chooses a production rule for a non-terminal based on the current input symbol.



Types of Top-Down Parsers

Type	Description	Mechanism	Drawback
Recursive Descent Parser	A collection of mutually recursive functions, where each function corresponds to a non-terminal in the grammar.	The functions directly implement the production rules. It may use backtracking (trying one rule, failing, and then undoing the choice to try another).	Inefficient due to potential backtracking, which can lead to exponential time complexity for some grammars.
Predictive Parser (e.g., LL(1))	A special case of recursive descent that does not require backtracking. It uses a lookahead terminal to deterministically choose the correct production rule.	Uses a stack and a Parsing Table (built using FIRST and FOLLOW sets) to make decisions with 1 symbol of lookahead.	Requires the grammar to be in a special form (LL(1) Grammar).



LL(1) parsing

LL(1) parsing is a method of Non-Recursive Top-Down Parsing that operates without the need for backtracking. It is deterministic, meaning it can always predict the next production rule based on the current non-terminal and one symbol of lookahead.

1. The LL(1) Notation Explained

The name "LL(1)" describes the parser's action:

L (First L): The input is scanned from Left to right.

L (Second L): A Leftmost derivation is produced.

1: The parser uses 1 symbol of lookahead to make its decision.



The Four Core Components

The LL(1) parsing system consists of four main parts that work together:

1. **Input Buffer:** Holds the input string (a sequence of tokens) to be parsed, always terminated by the special end-marker symbol, \$.
2. **Stack:** The working memory of the parser. It holds grammar symbols (terminals and non-terminals). It is initialized with the end-marker \$ at the bottom and the **Start Symbol** (\$) on top: [\$\$].
3. **Parsing Program:** The control unit that determines the action based on the top of the stack and the current lookahead symbol.
4. **LL(1) Parsing Table (\$M\$):** A two-dimensional array where:
 - **Rows** are indexed by **Non-terminals (\$A\$)**.
 - **Columns** are indexed by **Terminals (\$a\$)** (including \$).
 - An entry $SM[A, a]$ contains the production rule $SA \rightarrow \alpha$ that should be applied.



Requirements and Limitations

LL(1) parsing is only possible if the grammar satisfies two strict conditions, ensuring the parsing table is deterministic (i.e., every cell has at most one production):

1. **No Left Recursion:** The grammar must be free of both direct and indirect left recursion. This is solved by converting the grammar to an equivalent **right-recursive** form.
2. **No Left Factoring Ambiguity:** For any non-terminal A , if it has multiple alternatives, the FIRST sets of the alternatives must be disjoint. This is solved by **left factoring** the grammar.